

JavaScript Modules – Import/Export & Circular Dependencies (Full Explanation)

What is Import/Export?

JavaScript modules allow you to split your code into multiple files. You can export functions or variables from one file and import them into another file.

Working Example

```
// utils.js
export function sum(a, b) {
  return a + b;
}

// test.js
import { sum } from "./utils.js";

console.log(sum(2, 3));

// index.js
import "./test.js";
```

Explanation: utils.js exports a function, test.js imports and uses it, and index.js starts the application.

What is a Circular Dependency?

A circular dependency happens when two files import each other. This can cause problems because one file may not be fully loaded when the other tries to use it.

Bad Example (Circular Dependency)

```
// utils.js
import { test } from "./test.js";

export function sum(a, b) {
  return a + b;
}

// test.js
import { sum } from "./utils.js";

export function test() {
  console.log(sum(2, 3));
}
```

Problem: utils.js loads test.js, and test.js loads utils.js again. One of them will be incomplete.

Diagram – Circular Dependency

```
utils.js ↔ test.js
  ■ loop / undefined values
```

Correct Structure (Best Practice)

```
index.js → test.js → utils.js
```

Always keep imports in one direction to avoid problems.

Key Rules

- Use export/import freely
- Avoid circular dependencies
- Keep a clear one-direction flow

- Put shared logic into separate files

Interview Tip

Circular dependencies can cause modules to be partially initialized, leading to undefined values or runtime errors.